第 6 章教学源码与变体

以下由正式文件生成。主程序、同名类型、服务与命名模块的输出目录分开，完整命令见运行说明。

CH06-E01 · 加载器主程序

src/main/java/org/example/jvminternals/ch06/LoaderJourney.java

```
package org.example.jvminternals.ch06;

import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;

/** CH06-E01：默认委派下的单名定义，展示同名类型、签名和已授权转换失败。 */
public final class LoaderJourney {
    private static final System.Logger LOGGER =
System.getLogger(LoaderJourney.class.getName());
    private static final String PLUGIN = "org.example.jvminternals.ch06.isolated.Plugin";

    private LoaderJourney() {
    }

    /** 接收本章 Plugin 类文件，所有输入均为本地已编译教学产物。 */
    public static void main(String[] args) throws IOException,
ReflectiveOperationException {
        if (args.length != 1) {
            throw new IllegalArgumentException("用法：LoaderJourney <Plugin 类文件>");
        }
        LOGGER.log(System.Logger.Level.INFO, "开始读取加载器身份并定义两份同名类型");
        System.out.println("Object.loader=" + loaderName(Object.class.getClassLoader()));
        Class<?> sqlDriver = Class.forName("java.sql.Driver");
        System.out.println("Driver.loader=" + loaderName(sqlDriver.getClassLoader()));
        System.out.println("Driver.module=" + sqlDriver.getModule().getName());
        System.out.println("application.loader=" +
loaderName(LoaderJourney.class.getClassLoader()));
        System.out.println("object.resource=" +
Object.class.getResource("/java/lang/Object.class"));
        System.out.println("application.resource=" +
LoaderJourney.class.getClassLoader().getResource("notes.txt"));
        SingleNameLoader left = new SingleNameLoader("left", Path.of(args[0]));
        SingleNameLoader right = new SingleNameLoader("right", Path.of(args[0]));
        Class<?> leftType = Class.forName(PLUGIN, false, left);
        Class<?> rightType = Class.forName(PLUGIN, false, right);
        Object leftObject = leftType.getConstructor().newInstance();
        Object rightObject = rightType.getConstructor().newInstance();
```

```
        System.out.println("same.binaryName=" +
leftType.getName().equals(rightType.getName()));
        System.out.println("same.type=" + (leftType == rightType));
        System.out.println("left.echo.parameter.loader=" + loaderName(
                leftType.getMethod("echo", leftType).getParameterTypes()
[0].getClassLoader()));
        System.out.println("right.echo.parameter.loader=" + loaderName(
                rightType.getMethod("echo", rightType).getParameterTypes()
[0].getClassLoader()));
        System.out.println("left.echo.result.loader=" +
loaderName(leftType.getMethod("echo", leftType)
                .invoke(leftObject, leftObject).getClass().getClassLoader()));
        try {
            leftType.cast(rightObject);
        } catch (ClassCastException exception) {
            // 用户已明确授权的隔离转换失败；记录异常数据，不添加断言。
            System.out.println("cross.cast=" + exception.getClass().getSimpleName() + ": "
+ exception.getMessage());
        }
        ClassLoader relay = new ClassLoader("relay", left) {
            // 只使用继承的默认委派，不另行定义 Plugin。
        };
        Class<?> requested = Class.forName(PLUGIN, false, relay);
        System.out.println("request.loader=" + relay.getName());
        System.out.println("request.result.definer=" +
loaderName(requested.getClassLoader()));
        System.out.println("relay.same.as.left=" + (requested == leftType));
        LOGGER.log(System.Logger.Level.INFO, "加载器身份观察结束");
    }

    private static String loaderName(ClassLoader loader) {
        return loader == null ? "bootstrap(null)" : loader.getName();
    }

    /** 不覆盖 loadClass，仅在平台加载器找不到目标时由 findClass 提供一份定义。 */
    private static final class SingleNameLoader extends ClassLoader {
        private final Path classFile;

        private SingleNameLoader(String name, Path classFile) {
            super(name, ClassLoader.getPlatformClassLoader());
            this.classFile = classFile;
        }

        @Override
        protected Class<?> findClass(String name) throws ClassNotFoundException {
            if (!PLUGIN.equals(name)) {
                throw new ClassNotFoundException(name);
            }
            try {
                byte[] bytes = Files.readAllBytes(classFile);
                return defineClass(name, bytes, 0, bytes.length);
```

```
            } catch (IOException exception) {
                throw new ClassNotFoundException("无法读取教学类文件: " + classFile,
exception);
            }
        }
    }
}
```

CH06-E01 · 隔离 Plugin

variants/plugin/src/org/example/jvminternals/ch06/isolated/Plugin.java

```
package org.example.jvminternals.ch06.isolated;

/** 同一份字节由不同定义加载器加载，签名中的 Plugin 随各自命名空间解析。 */
public final class Plugin {
    /** 返回本加载器命名空间中的同类型对象。 */
    public Plugin echo(Plugin value) {
        return value;
    }
}
```

CH06-E02 · 服务发现

src/main/java/org/example/jvminternals/ch06/ServiceJourney.java

```
package org.example.jvminternals.ch06;

import java.io.IOException;
import java.net.URL;
import java.net.URLClassLoader;
import java.nio.file.Path;
import java.util.ServiceLoader;

/** CH06-E02: 临时设置上下文加载器，发现外部提供者并在结束时恢复。 */
public final class ServiceJourney {
    private static final System.Logger LOGGER =
System.getLogger(ServiceJourney.class.getName());

    private ServiceJourney() {
```

```
    }

    /** 接收仅含本章提供者与 META-INF/services 的编译目录。 */
    public static void main(String[] args) throws IOException {
        if (args.length != 1) {
            throw new IllegalArgumentException("用法：ServiceJourney <提供者输出目录>");
        }
        Thread current = Thread.currentThread();
        ClassLoader previous = current.getContextClassLoader();
        URL location = Path.of(args[0]).toUri().toURL();
        LOGGER.log(System.Logger.Level.INFO, "开始使用上下文加载器发现服务提供者");
        try (URLClassLoader providerLoader = new URLClassLoader("greeting-provider",
                new URL[]{location}, ClassLoader.getSystemClassLoader())) {
            current.setContextClassLoader(providerLoader);
            try {
                for (Greeter greeter : ServiceLoader.load(Greeter.class)) {
                    System.out.printf("provider=%s loader=%s service.loader=%s
message=%s%n",
                            greeter.getClass().getName(),
greeter.getClass().getClassLoader().getName(),
                            Greeter.class.getClassLoader().getName(), greeter.greet());
                }
            } finally {
                // 线程可能比本次发现过程更长寿，必须恢复原上下文引用。
                current.setContextClassLoader(previous);
            }
        }
        System.out.println("context.restored=" + (current.getContextClassLoader() ==
previous));
        LOGGER.log(System.Logger.Level.INFO, "提供者资源关闭，上下文已恢复；未声称类已卸载");
    }
}
```

共享服务接口

src/main/java/org/example/jvminternals/ch06/Greeter.java

```
package org.example.jvminternals.ch06;

/** CH06-E02：由父加载器共享的服务类型，提供者在隔离输出中实现。 */
public interface Greeter {
    /** 返回一条短教学问候。 */
    String greet();
}
```

服务提供者

variants/service/src/org/example/jvminternals/ch06/provider/FriendlyGreeter.java

```
package org.example.jvminternals.ch06.provider;

import org.example.jvminternals.ch06.Greeter;

/** 外部服务提供者，其 Greeter 类型由父加载器提供。 */
public final class FriendlyGreeter implements Greeter {
    @Override
    public String greet() {
        return "来自隔离提供者的问候";
    }
}
```

提供者配置

variants/service/resources/META-INF/services/org.example.jvminternals.ch06.Greeter

```
org.example.jvminternals.ch06.provider.FriendlyGreeter
```

CH06-E03 · 模块访问

variants/modules/src/lesson.client/org/example/jvminternals/ch06/client/ModuleAccessJourney.java

```
package org.example.jvminternals.ch06.client;

import java.lang.reflect.Method;
import org.example.jvminternals.ch06.moduleapi.PublicGreeting;

/** CH06-E03：已授权的模块访问失败与单进程导出 / 开放对照。 */
public final class ModuleAccessJourney {
    private static final System.Logger LOGGER =
System.getLogger(ModuleAccessJourney.class.getName());
    private static final String INTERNAL = "org.example.jvminternals.ch06.internal";
```

```
    private ModuleAccessJourney() {
    }

    /** 正常找到类后分别观察公开访问与私有成员深反射，不添加断言。 */
    public static void main(String[] args) throws ReflectiveOperationException {
        LOGGER.log(System.Logger.Level.INFO, "开始观察命名模块的访问边界");
        System.out.println("api=" + PublicGreeting.message());
        Class<?> secret = Class.forName(INTERNAL + ".Secret");
        Module caller = ModuleAccessJourney.class.getModule();
        Module library = secret.getModule();
        System.out.println("class.found=" + secret.getName());
        System.out.println("caller.reads.library=" + caller.canRead(library));
        System.out.println("package.exported=" + library.isExported(INTERNAL, caller));
        System.out.println("package.open=" + library.isOpen(INTERNAL, caller));
        Method publicMethod = secret.getMethod("message");
        try {
            System.out.println("public.invoke=" + publicMethod.invoke(null));
        } catch (IllegalAccessException exception) {
            // 访问失败属于本章获授权案例，不等于类没有找到。
            System.out.println("public.invoke=" + exception.getClass().getSimpleName());
        }
        Method privateMethod = secret.getDeclaredMethod("hidden");
        boolean accessible = privateMethod.trySetAccessible();
        System.out.println("private.accessible=" + accessible);
        if (accessible) {
            System.out.println("private.invoke=" + privateMethod.invoke(null));
        }
        LOGGER.log(System.Logger.Level.INFO, "命名模块访问观察结束");
    }
}
```

调用模块声明

variants/modules/src/lesson.client/module-info.java

```
/** 显式读取 lesson.library，未因此获得内部包访问权。 */
module lesson.client {
    requires lesson.library;
}
```

库模块声明

variants/modules/src/lesson.library/module-info.java

```
/** 只导出公开接口包，内部包默认既不导出也不开放。 */
module lesson.library {
    exports org.example.jvminternals.ch06.moduleapi;
}
```

模块公开类型

variants/modules/src/lesson.library/org/example/jvminternals/ch06/moduleapi/PublicGreeting.java

```
package org.example.jvminternals.ch06.moduleapi;

/** 命名模块中正常可访问的公开入口。 */
public final class PublicGreeting {
    private PublicGreeting() {
    }

    /** 返回本模块的公开信息。 */
    public static String message() {
        return "公开包调用成功";
    }
}
```

模块内部类型

variants/modules/src/lesson.library/org/example/jvminternals/ch06/internal/Secret.java

```
package org.example.jvminternals.ch06.internal;

/** 类本身是 public，包是否导出或开放仍由模块规则决定。 */
public final class Secret {
    private Secret() {
    }
```

```
    /** 用于普通跨模块访问检查的公开方法。 */
    public static String message() {
        return "内部包公开成员调用成功";
    }

    private static String hidden() {
        return "深反射读取成功";
    }
}
```

CH06-E04 · 子模块层

src/main/java/org/example/jvminternals/ch06/LayerJourney.java

```
package org.example.jvminternals.ch06;

import java.lang.module.Configuration;
import java.lang.module.ModuleFinder;
import java.nio.file.Path;
import java.util.Set;

/** CH06-E04：从本地模块创建子层，将模块关系与加载器身份分别观察。 */
public final class LayerJourney {
    private static final System.Logger LOGGER =
System.getLogger(LayerJourney.class.getName());

    private LayerJourney() {
    }

    /** 接收含 lesson.library 的编译目录，不使用远程模块。 */
    public static void main(String[] args) throws ReflectiveOperationException {
        if (args.length != 1) {
            throw new IllegalArgumentException("用法：LayerJourney <命名模块输出目录>");
        }
        LOGGER.log(System.Logger.Level.INFO, "开始创建一个本地子模块层");
        ModuleFinder finder = ModuleFinder.of(Path.of(args[0]));
        Configuration configuration = ModuleLayer.boot().configuration()
                .resolve(finder, ModuleFinder.of(), Set.of("lesson.library"));
        ModuleLayer layer = ModuleLayer.boot().defineModulesWithOneLoader(configuration,
                ClassLoader.getSystemClassLoader());
        Module library = layer.findModule("lesson.library").orElseThrow();
        Class<?> type = layer.findLoader("lesson.library")
                .loadClass("org.example.jvminternals.ch06.moduleapi.PublicGreeting");
        System.out.println("caller.named=" + LayerJourney.class.getModule().isNamed());
        System.out.println("library.in.boot=" +
```

```
ModuleLayer.boot().findModule("lesson.library").isPresent());
        System.out.println("library.in.child=" + (library.getLayer() == layer));
        System.out.println("class.module=" + type.getModule().getName());
        System.out.println("class.loader.type=" +
type.getClassLoader().getClass().getName());
        System.out.println("public.result=" + type.getMethod("message").invoke(null));
        LOGGER.log(System.Logger.Level.INFO, "子层观察结束");
    }
}
```